

Exercícios de PySpark

15 exercícios aplicados aos arquivos CSV clientes_spark e vendas_spark

Base de dados

Os exercícios devem ser resolvidos a partir da leitura dos arquivos CSV. Utilize o Google Colab e exporte o notebook como .py.

Arquivo	Colunas
clientes_spark.csv	id_cliente, cliente, estado, cidade, data_cadastro
vendas_spark.csv	id_venda, id_cliente, data_venda, produto, categoria, estado, quantidade, preco_unitario, valor_total

Observação: a tabela de vendas já traz o id_cliente. Por isso não existe a tabela intermediária vendas_clientes do notebook, e o join é direto entre vendas e clientes.

Bloco 1: Leitura e exploração

Exercício 1: Leitura dos arquivos

Leia os dois arquivos CSV com header e inferSchema ativados, usando spark.read. Armazene em df_vendas e df_clientes. Exiba as 10 primeiras linhas de cada um e o schema (printSchema). Em seguida, informe quantos registros cada DataFrame possui.

Exercício 2: Conferindo os tipos

Verifique no schema o tipo inferido das colunas data_venda e data_cadastro. Se elas vierem como string, converta-as para o tipo date usando to_date e withColumn. Exiba o schema novamente para confirmar a conversão.

Bloco 2: Select e filtros (where)

Exercício 3: Seleção de colunas

Do df_vendas, selecione apenas produto, categoria, estado e preco_unitario e exiba 15 linhas.

Exercício 4: Filtro simples

Liste todas as vendas em que a quantidade seja igual a 10. A partir desta lista, exiba a quantidade total de linhas retornadas.

Exercício 5: Filtro com múltiplas condições

Liste as vendas cujo preco_unitario seja maior ou igual a 200 e menor que 3000, e cujo estado seja "SP". Exiba apenas id_venda, produto, estado e preco_unitario.

Exercício 6: Filtro por categoria

Exiba os campos produto e categoria das vendas da categoria "Eletronicos". Use `distinct()` para listar cada combinação uma única vez.

Dica: antes, descubra quais categorias existem com `select("categoria").distinct()`.

Bloco 3: Colunas calculadas

Exercício 7: Validando o valor total

Crie a coluna `valor_calculado = quantidade * preco_unitario` com `withColumn`. Compare com a coluna `valor_total` já existente no arquivo e verifique se existe alguma linha em que os dois valores difiram (considere uma tolerância de 0,01). Informe quantas linhas divergem.

Exercício 8: Extração de datas

Usando `year` e `month` (de `pyspark.sql.functions`), crie as colunas `ano_venda` e `mes_venda` a partir de `data_venda`. Exiba 10 linhas com `id_venda`, `data_venda`, `ano_venda` e `mes_venda`.

Bloco 4: Agregações (groupBy)

Exercício 9: Total por categoria

Agrupe as vendas por categoria e calcule o total vendido (sum de `valor_total`), com o alias `total_categorizado`. Ordene do maior para o menor.

Exercício 10: Resumo estatístico por categoria

Para cada categoria, calcule em uma única agregação:

- a quantidade de vendas (count de `id_venda`);
- a quantidade total de itens vendidos (sum de `quantidade`);
- o valor total vendido (sum de `valor_total`);
- o valor médio por venda (avg de `valor_total`), arredondado com `round` para 2 casas.

Exercício 11: Agregação por duas dimensões

Calcule o valor total vendido por estado e categoria. Exiba os 10 maiores resultados e informe qual combinação estado/categoria teve a maior receita.

Bloco 5: Spark SQL

Exercício 12: Consulta SQL

Registre o `df_vendas` como view temporária chamada `vendas` (`createOrReplaceTempView`). Com `spark.sql`, escreva uma consulta que retorne, por produto, o total vendido e a quantidade de vendas, mostrando apenas os produtos com total vendido acima de 1.000.000 (use `HAVING`), ordenados do maior para o menor.

Bloco 6: Joins

Exercício 13: Inner join

Faça um `inner join` entre `df_vendas` e `df_clientes` pela coluna `id_cliente`. Selecione `id_venda`, `cliente`, `cidade`, `produto` e `valor_total`, referenciando as colunas pelos DataFrames de origem (por exemplo, `df_clientes["cliente"]`) para evitar ambiguidade. Compare a contagem de linhas do resultado com a de `df_vendas` e explique se houve perda de registros e por quê.

Exercício 14: Left join e clientes sem compras

Faça um `left join` partindo de `df_clientes` para `df_vendas`. Identifique os clientes que **nunca compraram** (aqueles em que `id_venda` é nulo, usando `isNull()`) e informe quantos são. Exiba 10 deles com `id_cliente`, `cliente` e `data_cadastro`.

Exercício 15: Join + agregação (ranking)

Una `vendas` e `clientes` e calcule o **Top 10 clientes** por valor total comprado. O resultado deve conter `id_cliente`, `cliente`, `cidade`, a quantidade de compras e o valor total gasto (arredondado em 2 casas).